# 設計メモ

Author: yuichi(作成 2026-10-10 / 更新 2026-10-21)

関数ごとの役割は functions.md、AT コマンドは at-command-table.md を参照してください。

## データの流れ

```
Monitor.bat ─► Monitor.ps1
                  │  起動時: 二重起動の確認（動いていれば監視画面を前面に出して終了）
                  │          監視画面を開く（Browser.ps1）
                  │
                  └─(1 秒ごとにループ)─┐
                     ├─ 時刻が $CallTimes に一致 → AtIo.exe ... "ATD<番号>;" <秒> "ATH"
                     └─ 周期ごと                 → AtIo.exe ... "AT+CSQ" / "AT+CREG?" / $Extras
                           │
                           ├─► logs/csq_YYYYMM.csv       追記（月別。取得結果）
                           ├─► logs/test_YYYYMMDD.log    追記（日別。試験ログ）
                           └─► data.js                   毎回まるごと書き換え（一時ファイル → 置換）

index.html ──(3 秒ごと)── <script src="data.js?t=…"> を差し替えて読み込み → カードとグラフを更新
```

## 設計判断

### HTTP サーバを使わない

画面は `file://` で開く静的な HTML です。データは JSON を `fetch` せず、**`<script>` タグで `data.js` を読み込みます**。 `fetch` は `file://` では CORS 制約で失敗しますが、 `<script>` の読み込みは制約を受けません。 HttpListener による常駐サーバ(ブロッキング、CORS、エンコードの問題)が丸ごと不要になります。 URL にクエリ( `?t=…` )を付けてキャッシュを避けています。Edge・Chrome・Firefox のどれでも同じように動きます。

### C# は I/O だけ

`AtIo.exe` は `AtIo.exe <Port> <BaudRate> <Cmd> [WaitSec] [AfterCmd]` で動き、応答を標準出力へそのまま出します。 コマンドの種類・応答の解釈・周期などの知識は持ちません。**C# を書き換える必要が出るのは、シリアル通信そのものを変えるときだけ**にしています。

- 接続は 8N1、DTR/RTS を ON にして開き、開いた直後に `ATE0V1` (エコー OFF・文字応答)を送ってから、その応答を捨てます。
- `AfterCmd` なし:応答の終端行( `OK` / `ERROR` / `NO CARRIER` / `+CME ERROR: n` など)を受信した時点で終了。 `WaitSec` は上限です。終端は、これまでに受信した全文に対して判定するので、応答が途中で分割されても取りこぼしません。
- `AfterCmd` あり: `WaitSec` 秒間受信を続け、その後 `AfterCmd` を送ります。試験発信( `ATD…;` → 呼出 → `ATH` )用です。
- 終了コード: `0` 正常、 `1` 引数不足、 `2` I/O エラー(エラー内容は標準エラー出力に `IO Error: …` で出す)。

### COM ポートは開きっぱなしにしない

`AtIo.exe` は AT コマンド 1 回ごとに起動し、COM ポートを「開く → 送受信 → 閉じる」で終わります(1 回の取得で `AT+CSQ` と `AT+CREG?` の 2 回)。 周期の合間はポートが空くので、プロセスがポートを握り続けることがありません。 試験発信だけは、 `ATD` から `ATH` までを 1 回の起動の中で行い、通話中はポートを開いたままにします。

## 取得のリトライはしない

`AtIo.exe` が失敗しても、その周期では再送しません。失敗として CSV と試験ログに記録し、次の周期(既定 180 秒後)に取り直します。 リトライを入れると失敗が見えにくくなるため、まず試験ログで失敗の頻度と傾向を確かめる方針にしています(警報のための「連続失敗回数」の考え方は今後の課題)。

## 収集ループは 1 秒刻み

取得周期(既定 180 秒)の間も、1 秒ごとに時刻を確認します。定時発信の `HH:mm` を取りこぼしません。 同じ分に 2 度発信しないよう、 `yyyyMMdd_HHmm` をキーにして制御しています。発信の直後には、すぐに 1 回取得します。 次の取得は「前回の取得が終わってから `$PollSec` 秒後」です。取得にかかる時間の分だけ、時刻は少しずつ後ろにずれます。

## 入口は Monitor.bat ひとつ(二重起動の防止)

輪番で運用する人が覚えることを減らすため、入口を `Monitor.bat` ひとつにしています。

- 起動時に名前付き Mutex( `Global\CsqWatch` 。 `-Mock` は `Global\CsqWatch_Mock` )を取ります。 `Global` 名前空間なので、別のユーザーでログオンして起動した場合も検出できます。
- 取れなかったとき(すでに動いているとき)は、監視を始めずに監視画面を前面に出し(閉じていれば開き)、5 秒後に終了コード `3` で終わります。 `Monitor.bat` は終了コード `3` のときだけ `pause` せずに閉じます。
- 前回の Monitor が異常終了していた場合は、Mutex の放棄( `AbandonedMutexException` )を受け取って普通に起動し、試験ログに WARN を残します。
- 動いている Monitor の情報(PID・開始時刻・ユーザー・PC 名)は `logs\monitor_running.txt` に書き、Ctrl+C で止めたときに消します。

## 社外監視の窓を最優先にする

この PC では、社外監視(VPN 経由の衛星事業者のダッシュボード)の窓が最優先です。監視画面はそれに影響を与えないように開きます( `Browser.ps1` )。

- 社外監視とは別のブラウザを選びます( `$Browser = 'auto'` は Chrome → Edge の順。Firefox は他業務で使うので `auto` に含めない)。
- 専用プロファイル( `app\profile_<ブラウザ名>` )で起動するので、社外監視のブラウザとはプロセスが分かれます。こちらが重くなっても落ちても巻き込まず、Cookie やログイン情報も共有しません。
- Chrome・Edge はアプリモード( `--app` )で開き、タブのない専用の窓にします。
- すでに開いていれば、専用プロファイルで動いているブラウザの本体プロセスを探し、その**プロセス ID を指定して**前面に出します( `WScript.Shell` の `AppActivate` )。窓のタイトルで探さないので、社外監視の窓を動かすことはありません。
- Monitor は CPU 優先度「通常以下」( `BelowNormal` )で動き、子プロセス( `AtIo.exe` ・監視画面のブラウザ)もそれを引き継ぎます。
- Monitor と監視画面はローカルのファイルだけで動き、ネットワークを使いません。

## 取得失敗と圏外を分ける

端末が値を返さなかったこと(COM のエラー・無応答)と、端末が「圏外・測定できない( `99` )」と返したことは、運用上の意味がまったく違います。

| 状況 | `CsqOk` | 画面 | グラフ | CSV の 3 列目 |
|---|---|---|---|---|
| 端末が値を返した | `true` | 値と `値 / 最大値` | その値 | `値 / 最大値` |
| 端末が `99` を返した | `true` | `圏外 (99)` | 0 | `圏外` |
| 取得失敗 | `false` | `--` と赤の `取得失敗` | 線が途切れる( `null` ) | `取得失敗` |

### 信号の目盛りは機器ごとに設定する

`AT+CSQ` の rssi の範囲は機器によって違います(GSM は 0〜31、IsatPhone 2 は 0〜55)。GSM の dBm 換算( `-113 + 2 × rssi` )は GSM 以外では意味を持たないので使わず、生の値を `値 / 最大値` で表示します。最大値は `Config.ps1` の `$RssiMax` で、グラフの縦軸もこれに合わせます。

### 更新停止の検知

`data.js` に取得周期( `intervalSec` )を含め、画面側で最終更新時刻と比べます。周期の 2 倍 + 10 秒を超えると STALE 表示にします。 `Monitor` の停止、COM の切断、 `data.js` が出力されないとき、のいずれにも気づけます。

### 試験ログ

不具合の解析とデータ取得の検証のため、 `logs\test_YYYYMMDD.log` に 1 行ずつ記録します。書式と見かたは README の「試験ログ」を参照してください。

- 1 行ごとにファイルを開いて追記し、閉じます。強制終了や PC の再起動の直前まで残ります。
- 応答はログ用に整形します(PowerShell 5.1 が外部プログラムのエラー出力に付ける「発生場所 ...」などの飾りを除く)。判定には整形前の応答を使います。
- メインループを `try { } finally { }` で囲み、Ctrl+C で止めたときに `STOP` を残します。× ボタンで閉じた・再起動・強制終了では残らないので、「 `STOP` がない = 外部要因」と見分けられます。
- 予期しないエラーは `trap` で `ERR UNHANDLED` (行番号付き)を残してから止まります。

## data.js の形式

```
window.CSQ_DATA = {
  "mock": false,
  "intervalSec": 180,
  "rssiMax": 55,                    // Config.ps1 の $RssiMax (グラフの縦軸の最大値)
  "deviceName": "IsatPhone 2",      // Config.ps1 の $DeviceName (画面の見出し)
  "current": {
    "LastUpdate": "2026-10-04 08:00:03",
    "CsqOk": true,                  // false = 端末から値を取れなかった (画面は「取得失敗」)
    "Rssi": 37,                     // AT+CSQ の <rssi> (IsatPhone 2 は 0-55, 99=圏外)
    "Dbm": "37 / 55",               // 画面の Signal Level。生の値 / $RssiMax (GSM の dBm
換算は使わない)。失敗時は "取得失敗"
    "Ber": 0,
    "RegStatus": "同期完了 (Registered)",
    "Extras": [ { "name": "Operator", "value": "..." } ]   // Config.ps1 の $Extras
  },
  "history": [ { "t": "2026-10-04 08:00:03", "r": 37 } ]   // 直近 $HistoryMax 点。端末の圏
外(99)は 0、取得失敗は null (グラフが途切れる)
};
```

`Dbm` という名前は、以前 dBm を入れていたときの名残です(画面との互換のため名前は変えていません)。

## CSV の形式

`logs/csq_YYYYMM.csv` (UTF-8 BOM 付き、Excel でそのまま開けます)

```
Timestamp,RSSI,Level,BER,RegStatus,Notes
2026-10-04 08:00:03,37,37 / 55,0,同期完了 (Registered),Periodic
2026-10-04 08:03:03,99,取得失敗,99,ポートエラー,IO Error: 書き込み操作がタイムアウトしました。
2026-10-04 08:00:00,,,,,TestCall +81...        ← 試験発信の記録
```

- 取得に失敗した行は、3 列目が `取得失敗` (2026-10-21 より前は `N/A` )、2 列目が `99` です。端末が返した圏外( `99` )とは 3 列目で見分けます。
- 3 列目は 2026-10 までは `dBm` (GSM の換算値)でした。Inmarsat の rssi には使えないため、 `Level` (生の値 / 最大値)に変えています。古い CSV の `dBm` 列の値は意味を持たないので、分析には 2 列目の `RSSI` を使ってください。
- `Notes` には、 `Periodic` 、 `CSQ no response` 、COM エラーのメッセージ(1 行目)などが入ります。
- 旧名(IsatMon)のころの CSV は `isat_YYYYMM.csv` です。新しい CSV がまだない月は、起動時に旧 CSV からグラフを復元します(読むだけで、書き換えません)。

## 起動時に作られるもの

| もの | 場所 | 内容 |
|---|---|---|
| `Config.ps1` | `app\` | 初回に `Config.sample.ps1` をコピー |
| `data.js` | `app\` | 画面用のデータ |
| `csq_YYYYMM.csv` | `app\logs\` | 取得結果(月別) |
| `test_YYYYMMDD.log` | `app\logs\` | 試験ログ(日別) |
| `monitor_running.txt` | `app\logs\` | 動いている Monitor の情報 |
| `profile_<ブラウザ名>\` | `app\` | 監視画面用のブラウザプロファイル |

いずれも `.gitignore` で Git の管理外にしています。

## 文字コードの規約

`.ps1` `.cs` `probe_commands.txt` は **UTF-8(BOM 付き)・CRLF** です( `.editorconfig` と `.gitattributes` で固定)。BOM がないファイルは、日本語環境の Windows PowerShell 5.1 と `csc.exe` が ANSI(cp932)と誤読し、日本語の文字列が壊れます。 `.bat` は cmd が cp932 を使うため、**ASCII のみ**にしています(日本語を入れないでください)。 `.md` は UTF-8(BOM なし)・LF です。

## 補足

- `Common.ps1` の `Invoke-At` は PowerShell から `AtIo.exe` を呼びます。コマンド文字列に `"` を含むもの( `AT+CGDCONT=1,"IP","apn"` など)は、PowerShell 5.1 のネイティブコマンド引数の扱いで壊れるため、現状は送れません。
- `-Mock` の疑似応答は `Common.ps1` の `Get-MockResponse` にあります。 `$Extras` で新しいコマンドを試すときは、ここに疑似応答を足すと画面で確認できます。疑似データは IsatPhone 2 の実測に近い範囲(rssi 20〜50)で動きます。
- 実行環境は Windows 11 標準の PowerShell 5.1・.NET Framework 4.x・Edge だけです。インストールも管理者権限も要りません。